

Diplomatura

Inteligencia Artificial

Módulo 5:

Inteligencia Artificial Generativa

Unidad 3:

Modelos Generativos en el Lenguaje - Parte 02



Presentación

Esta unidad se adentra en el corazón de la inteligencia artificial generativa contemporánea, donde el lenguaje ya no es solo medio de interacción, sino herramienta de diseño. Comenzamos explorando la ingeniería de prompts y su lógica interna: cómo construir instrucciones claras, qué efectos tienen el contexto y el rol, y por qué ajustar un prompt puede transformar por completo el comportamiento de un modelo.

Luego pasamos a los modelos de razonamiento, una nueva generación de sistemas capaces de descomponer problemas, verificar hipótesis y ajustar su proceso según la dificultad. Comprender esta diferencia —entre predecir texto y razonar estructuralmente— es clave para elegir el modelo adecuado en tareas complejas.

La unidad continúa con los agentes de IA, sistemas que combinan memoria, herramientas y planificación para ejecutar acciones reales: consultar datos, operar aplicaciones o coordinar tareas. El Model Context Protocol aparece como el puente que busca estandarizar esta interacción, permitiendo que distintos agentes y plataformas trabajen de forma interoperable.

Finalmente, abordamos los workflows agénticos, patrones que organizan cómo un agente piensa, actúa y se corrige. Estos modelos ya están transformando procesos en soporte técnico, análisis de datos y automatización empresarial.



Bloques temáticos

1. Ingeniería de Prompts Avanzados.
2. Modelos LLM avanzados y capacidades de razonamiento.
3. Agentes de Inteligencia Artificial: Definición. Memoria de corto y largo plazo. Herramientas y funciones. MCP (Model Context Protocol).
4. Workflows Agenticos. Niveles de Autonomía. Casos de Aplicación.

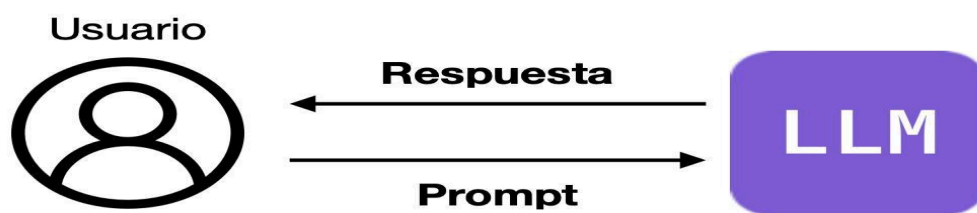
Ingeniería de Prompts Avanzados

¿Qué es un Prompt?

Un prompt es una entrada que le das a un modelo de lenguaje, como parte del aprendizaje automático. Al trabajar con modelos como GPT-5, Claude, Gemini u otros modelos avanzados, el prompt es la instrucción, consulta o contexto que orienta al modelo para generar una respuesta o completar una tarea.

Los prompts pueden adoptar diferentes formas: una pregunta directa, una declaración a completar, o una serie de instrucciones detalladas. En cualquier caso, el objetivo principal es guiar al modelo para que entienda y resuelva una tarea de manera precisa y relevante.

Ejemplo: Imagina que quieres que un modelo genere ideas para nombres de empresas; el prompt podría ser: “Genera 10 ideas creativas para nombres de una nueva empresa de tecnología enfocada en IA.”



Aspectos clave de un Prompt

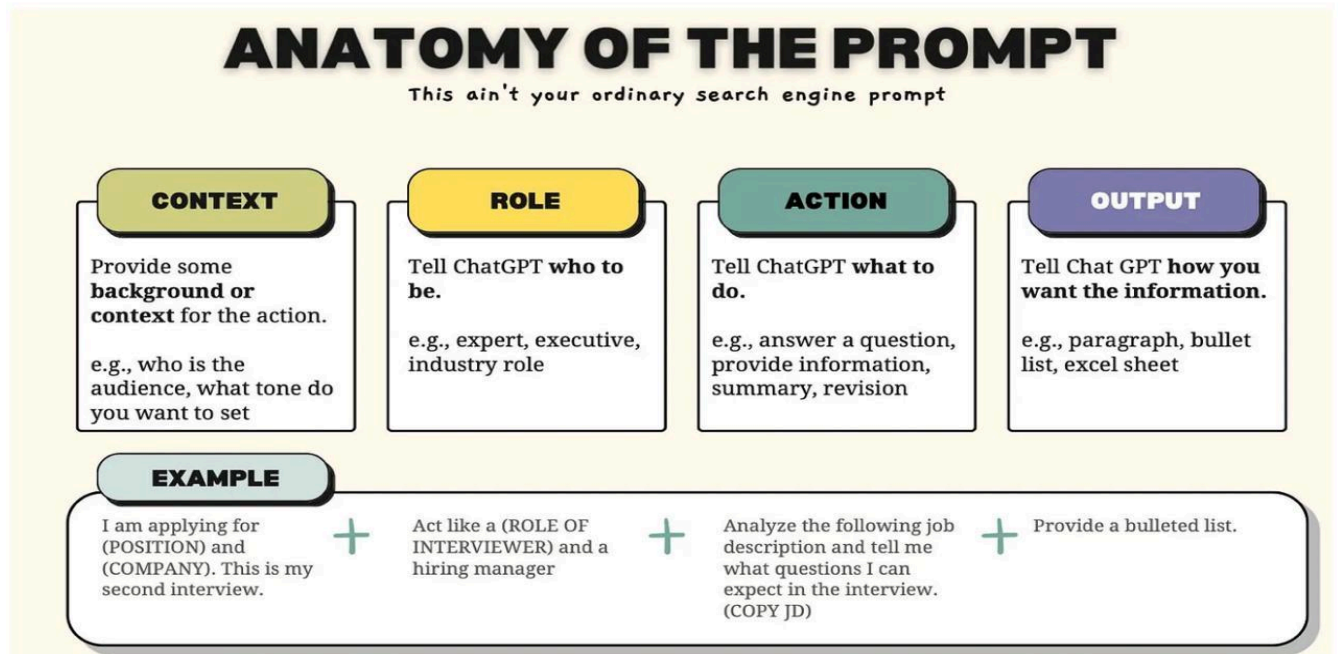


image source: <https://www.linkedin.com/in/rachel-rozen/>

Idea central: un buen prompt se arma con cuatro piezas que se encadenan en este orden: **Contexto + Rol + Acción + Output**.

1) Contexto (CONTEXT)

Dale al modelo el fondo de la situación para que entienda **de qué va** la tarea.

- ¿Quién es la audiencia? ¿Qué tono se espera?
- ¿Qué información previa o restricciones importan?

2) Rol (ROLE)

Indicá **quién** querés que sea el asistente. El rol define mirada, lenguaje y criterios.

- Ej.: “experto en X”, “ejecutivo”, “revisor técnico”, “entrevistador/hiring manager”.
- Sumá límites (“no inventes datos”, “seguí estilo institucional”).

3) Acción (ACTION)

Decile **qué** debe hacer, de forma concreta y operativa.

- Responder una pregunta, analizar un documento, generar opciones, resumir, revisar.
- Podés listar pasos y criterios de aceptación (p. ej., “10 preguntas + 2 follow-ups c/u”).

4) Salida esperada (OUTPUT)

Aclará **cómo** querés la respuesta.

- Formato: párrafo, lista con viñetas, tabla, JSON, etc.
- Detalles: límite de palabras, campos obligatorios, orden de presentación.

Cómo se combina (EXAMPLE)

Tal como en la imagen, el armado final es una suma de bloques:

[Contexto]

“Estoy aplicando a (PUESTO) en (EMPRESA). Es mi segunda entrevista.”

+ [Rol]

“Actuá como **entrevistador senior** y **hiring manager**.”

+ [Acción]

“Leé la descripción del puesto y decime **qué preguntas** puedo esperar en la entrevista.”

(pegá la descripción del puesto debajo)

+ [Output]

“**Devolvé una lista con viñetas.**”

Este enfoque no solo mejora la claridad y relevancia del prompt, sino que puede ser aplicado a diversos modelos de lenguaje para obtener respuestas óptimas. Además, recuerda que la forma en que estructuras tu prompt puede influir enormemente en la calidad de la respuesta generada por el modelo.

Ingeniería de Prompts

La Ingeniería de Prompts (o en inglés, prompt engineering) es el arte de diseñar, ajustar y refinar las instrucciones o “prompts” con el objetivo de obtener los mejores resultados posibles de un modelo de lenguaje. No se trata solo de escribir una consulta al azar; se requiere una cuidadosa planificación y un conocimiento profundo de cómo los modelos interpretan y responden a los diferentes tipos de prompts.

- Este proceso implica reescribir y ajustar continuamente los prompts para optimizar la calidad y relevancia de las respuestas generadas por el modelo. A través de iteraciones sucesivas, puedes afinar los resultados y orientar al modelo para cumplir con objetivos específicos.
- La ingeniería de Prompts es una combinación dinámica de creatividad y estrategia. Implica experimentar con diferentes formulaciones, probar cómo responde el modelo, y luego refinar esos prompts para obtener respuestas cada vez más precisas y útiles.

Sugerencia: Al trabajar con modelos multilingües, es importante recordar que un prompt en un idioma específico puede generar resultados diferentes en comparación con el mismo prompt en otro idioma, debido a las particularidades de interpretación lingüística del modelo.

Ventajas de la ingeniería de Prompts

- No requiere entrenamiento adicional: Lograr los resultados deseados no depende de reentrenar el modelo desde cero, lo que ahorra tiempo y recursos computacionales.
- Eficiente en costo y tiempo: La creación y ajuste de prompts es un proceso rápido que permite obtener resultados inmediatos. Esto resulta mucho más eficiente que la reconfiguración o el ajuste fino de los pesos internos del modelo.

- Flexibilidad de tareas: Los prompts pueden ser adaptados a una gran variedad de tareas, desde traducción y resumen de texto hasta generación de código y respuestas a preguntas de conocimiento general.
- Baja barrera de entrada: Incluso usuarios con conocimientos técnicos limitados pueden crear y utilizar prompts efectivos con solo aprender algunas reglas básicas de formulación.
- Prototipado rápido: Permite iterar y probar diferentes prompts en un corto periodo de tiempo para descubrir cuál produce las respuestas más adecuadas a tus necesidades. Esta capacidad de iterar rápidamente es especialmente útil en el desarrollo de chatbots, asistentes virtuales y otras aplicaciones conversacionales.
- Escalable: La misma técnica puede ser usada para trabajar en múltiples dominios, desde servicio al cliente hasta análisis financiero o educativo, simplemente ajustando los prompts para cada contexto.
- Aprovecha modelos preentrenados: La Ingeniería de Prompts hace uso de los conocimientos ya existentes en el modelo. Los modelos de lenguaje preentrenados tienen vastos conocimientos y habilidades, y un buen prompt puede aprovechar toda esa información sin la necesidad de entrenar de nuevo.

Nota adicional: Si bien la Ingeniería de Prompts puede ofrecer resultados impresionantes, es importante tener en cuenta que, en ciertos casos, un prompt mal formulado puede llevar a respuestas poco claras o irrelevantes. Por ello, se recomienda siempre probar múltiples variaciones y comparar resultados.

Desventajas de la ingeniería de Prompts

- Resultados inconsistentes: Incluso pequeños cambios en el prompt pueden conducir a resultados impredecibles o inesperados. Esto se debe a que los modelos de lenguaje, al ser sensibles a la formulación exacta de las palabras, pueden interpretar de manera diferente cada variación.

Consejo para mitigar: Realiza pruebas con varias versiones de un mismo prompt para identificar cuál genera respuestas más coherentes y útiles.

- Método de prueba y error: Se requiere experimentar con muchas iteraciones para encontrar el prompt óptimo. Esto puede consumir tiempo, especialmente en tareas complejas donde pequeños ajustes pueden llevar a grandes diferencias en la calidad de las respuestas.

Enfoque proactivo: Usa herramientas de automatización o frameworks especializados para probar rápidamente múltiples variaciones de prompts y evaluar sus resultados.

- Control limitado: A pesar de que se puede guiar al modelo con prompts, existe una falta de control fino sobre el comportamiento del modelo. No siempre es posible garantizar que el modelo responderá exactamente como se espera, lo que puede resultar en respuestas no deseadas o irrelevantes.

Solución alternativa: Si necesitas más control sobre el modelo, podría ser necesario hacer fine-tuning a un modelo específico para tu tarea en lugar de depender únicamente de prompts en un modelo preentrenado.

- Dificultad con Prompts complejos: A medida que las tareas se vuelven más específicas o complejas, diseñar prompts efectivos se convierte en un reto. Por ejemplo, si se espera que el modelo realice tareas que requieren razonamiento avanzado o múltiples pasos, un solo prompt podría no ser suficiente.

Recomendación: Divide la tarea en prompts más pequeños y manejables, y evalúa si puedes construir una “conversación” progresiva con el modelo para obtener la respuesta deseada.

- Problemas de escalabilidad: La Ingeniería de Prompts puede no escalar bien para aplicaciones altamente especializadas o para casos en los que se requieren respuestas extremadamente precisas. En aplicaciones donde se necesite mantener un alto grado de consistencia y exactitud, los prompts pueden quedarse cortos.

Alternativa escalable: Para tareas muy específicas, puede ser más efectivo desarrollar una arquitectura híbrida que combine prompts con otros enfoques, como ajuste fino del modelo (fine-tuning) o modelos complementarios que actúen como filtros adicionales para mejorar la calidad de la respuesta.

Técnicas de ingeniería de Prompts

Few-shot prompting

Consiste en proporcionar al modelo ejemplos específicos (normalmente de 2 a 5) de la tarea que quieres realizar. Esto le ayuda a entender el tipo de respuesta que desees.

Alcances: Los ejemplos proporcionan un contexto fuerte para el modelo, guiándolo a seguir un patrón preestablecido en sus respuestas. Esto es útil para tareas complejas o específicas como traducción, corrección de texto, o generación de código.

Limitaciones: La efectividad de los ejemplos depende de la calidad y relevancia de los mismos. Si los ejemplos no están bien elegidos, el modelo puede producir resultados inconsistentes. Además, proporcionar demasiados ejemplos puede limitar la creatividad del modelo o hacer que responda con menos diversidad.

Técnica ASPECT

ASPECT es un acrónimo que ayuda a diseñar prompts efectivos. La idea es asegurarse de que el prompt sea claro, conciso y específico.

- **A (Action):** Especifica la acción que el modelo debe realizar. Por ejemplo, “Explica”, “Resume”, “Genera”, etc.

- **S (Subject):** Define el tema o contexto del prompt. Esto ayuda a centrar la respuesta en el tema específico que necesitas.
- **P (Purpose):** Indica el propósito o la intención detrás del prompt. Por ejemplo, si quieres generar ideas creativas, opiniones, o explicar un concepto complejo.
- **E (Examples):** Proporciona ejemplos que el modelo puede seguir. Esto puede ser especialmente útil para guiar el formato o el tono de la respuesta.
- **C (Constraints):** Especifica cualquier restricción que debas tener en cuenta, como la longitud de la respuesta, el idioma, o el estilo.
- **T (Tone):** Define el tono deseado, como formal, técnico, conversacional, humorístico, etc.

Alcances: Esta técnica ayuda a diseñar prompts que sean más detallados y enfocados, lo que aumenta la probabilidad de obtener respuestas relevantes y útiles. Es especialmente efectiva cuando buscas respuestas específicas en lugar de generalidades.

Limitaciones: El proceso de construir prompts tan detallados puede consumir más tiempo, y no siempre garantiza que el modelo siga todas las instrucciones exactamente, especialmente si algunas de las restricciones son muy complejas.

Chain-of-thought prompting

Es una técnica que anima al modelo a “pensar en voz alta”, proporcionando razonamiento paso a paso antes de dar la respuesta final. Es especialmente útil para problemas que requieren lógica o análisis complejo.

Alcances: Mejora la capacidad del modelo para resolver tareas que requieren múltiples pasos o razonamiento lógico. Ayuda a reducir respuestas incorrectas en tareas como matemáticas, planificación o cualquier problema que requiera secuencia de ideas.

Limitaciones: Si el prompt no guía adecuadamente al modelo en el proceso de pensamiento, puede llevar a pasos de razonamiento incorrectos o irrelevantes. Además, puede resultar en respuestas largas y redundantes si no se estructura de manera adecuada.

Prompt layering (Prompt en capas)

Esta técnica implica dividir una tarea compleja en varios prompts más pequeños y secuenciales. Cada prompt se utiliza para obtener una parte específica de la información, que luego se combina para formar la respuesta final.

Alcances: Ayuda a mantener un control más preciso sobre la generación de la respuesta. Es útil para obtener detalles granulares sobre diferentes aspectos de una misma pregunta o para descomponer un problema en sus componentes esenciales.

Limitaciones: Este enfoque puede aumentar la complejidad del proceso y la duración de la interacción con el modelo. Además, puede requerir una planificación cuidadosa para asegurar que cada capa esté correctamente alineada con el resultado final deseado.

Iterative prompt refinement (Refinamiento iterativo)

Es la práctica de mejorar un prompt con base en las respuestas obtenidas. Si una respuesta no es lo suficientemente relevante, el prompt se ajusta y se vuelve a intentar, permitiendo que el proceso se redefine con cada iteración.

Alcances: Es una técnica que permite obtener resultados óptimos a través de un proceso continuo de prueba y ajuste. Útil para casos en los que se necesita un alto nivel de precisión o detalle en las respuestas.

Limitaciones: Este enfoque requiere tiempo y paciencia para iterar sobre los prompts y evaluar los resultados. Además, a veces puede ser difícil identificar exactamente qué parte del prompt necesita ajustes.

Estas técnicas te ayudarán a crear prompts más efectivos y precisos para cualquier modelo de lenguaje. Además, es importante recordar que experimentar con diferentes enfoques y combinaciones de estas técnicas puede llevar a obtener mejores resultados

Modelos LLM avanzados y capacidades de razonamiento

En los últimos dos años hemos sido testigos de un cambio importante en el desarrollo de modelos de lenguaje. Mientras que los LLM tradicionales (como

GPT- o Claude 2) generaban respuestas de forma casi instantánea basándose en patrones estadísticos, una nueva generación de modelos —conocidos como modelos de razonamiento o reasoning models— incorpora una etapa previa de "pensamiento" antes de entregar la respuesta final.

Esta no es simplemente una mejora incremental o un cambio de marketing: representa un cambio fundamental en cómo estos sistemas procesan las consultas complejas. Pensemos en la diferencia entre responder una pregunta de memoria versus resolver un problema matemático paso a paso: el primer enfoque es rápido pero puede fallar ante problemas complejos; el segundo requiere más tiempo pero ofrece mayor precisión y confiabilidad.

Ejemplo práctico: Si le pedimos a un LLM tradicional "calcula cuántos días laborables hay entre el 15 de marzo y el 30 de abril, excluyendo feriados nacionales de Argentina", es probable que intente dar una respuesta aproximada basada en patrones. Un modelo de razonamiento, en cambio, descompondrá el problema en pasos: (1) calcular días totales, (2) identificar fines de semana, (3) buscar feriados específicos, (4) restar y verificar el resultado.

Los modelos de razonamiento no reemplazan a los LLM convencionales; amplían el repertorio de herramientas disponibles. Para tareas que exigen lógica, cálculo, programación, análisis detallado o planificación, esta estrategia de "pensar primero, responder después" se traduce en respuestas más robustas y menos propensas a errores.

¿Qué entendemos por "razonamiento" en IA?

Cuando hablamos de "razonamiento" en el contexto de modelos de lenguaje, no nos referimos a pensamiento consciente como el humano. En cambio, nos referimos a un proceso estructurado de exploración y validación dentro del espacio de posibles respuestas.

Pensemos en cómo nosotros resolvemos un problema complejo:

- Planteamos el problema y lo descomponemos en partes
- Generamos hipótesis o soluciones parciales
- Evaluamos cada alternativa
- Descartamos opciones inconsistentes
- Verificamos la solución antes de considerarla final

Los modelos de razonamiento emulan este proceso. No "piensan" en sentido filosófico, pero sí gestionan mejor la búsqueda de soluciones: generan hipótesis parciales, las comparan, descartan las inconsistentes y validan las que superan criterios de calidad. Esto reduce la tendencia a dar respuestas impulsivas o puramente asociativas, típica de modelos que eligen la continuación más probable sin verificar si es correcta.

Tres principios fundamentales del razonamiento en IA

1. Estructuración en etapas

El problema se divide en fases: comprensión → desarrollo → verificación → respuesta. Similar a cómo un estudiante resuelve un ejercicio mostrando el procedimiento.

2. Escalado de cómputo adaptativo

El modelo invierte más recursos (tiempo y procesamiento) cuando detecta que la consulta es compleja. Para preguntas simples usa menos recursos; para problemas difíciles, más pasos de "pensamiento".

3. Uso integrado de herramientas

Durante el proceso de razonamiento, el modelo puede ejecutar cálculos, consultar bases de datos o verificar información, integrando estas acciones como parte natural de su proceso de pensamiento.

¿Cómo funcionan? Arquitectura y funcionamiento

Es importante aclarar que la arquitectura base no cambia radicalmente. Seguimos trabajando con grandes redes neuronales de tipo Transformer (la misma arquitectura que GPT, Claude, etc.), entrenadas con enormes cantidades de texto, código e información multimodal. La diferencia clave está en cómo se entrena y cómo se ejecuta el modelo.

Durante el entrenamiento: Aprender a razonar

Además de las técnicas tradicionales de entrenamiento, se incorpora algo llamado supervisión del proceso. En lugar de premiar únicamente la respuesta final correcta, se recompensa que el modelo construya trayectorias útiles hacia esa respuesta:

- Explicitar subpasos de solución
- Detectar inconsistencias en el razonamiento
- Reintentar cuando una vía no funciona
- Verificar resultados intermedios

Este enfoque —conocido como refuerzo del razonamiento— orienta al modelo a preferir rutas que dejan rastros de trabajo verificable y no atajos estadísticos que pueden parecer correctos pero son frágiles.

Durante la inferencia: El presupuesto de razonamiento

Cuando usamos un modelo de razonamiento, este no genera una única respuesta directa. En cambio, ejecuta un proceso de búsqueda que puede incluir:

- Generar múltiples rutas de solución en paralelo
- Comparar conclusiones entre diferentes aproximaciones (auto-consistencia)
- Realizar cálculos o verificaciones auxiliares
- Decidir cuándo detener la búsqueda (si hay suficiente confianza) o prolongarla (si el problema es muy complejo)

- Invocar herramientas externas cuando sea necesario (ejecutar código, consultar bases de datos, etc.)



Concepto clave: Presupuesto de razonamiento

El "presupuesto" es la cantidad de recursos (tiempo, pasos, muestras) que el sistema dedicará a pensar antes de responder. Un presupuesto bajo es rápido pero menos exhaustivo; un presupuesto alto es más lento pero más preciso. Muchos sistemas modernos ajustan este presupuesto automáticamente según la dificultad detectada de la consulta.

Transparencia controlada: La cadena de pensamiento

Una característica importante es que todo este proceso de "pensamiento" se gestiona de forma interna. Al usuario se le muestra:

- Un indicador de que el modelo está "razonando" (por ejemplo, un mensaje "Pensando...")
- La respuesta final
- Un resumen del proceso seguido (cuando es relevante o se solicita)

¿Por qué no mostrar todo el detalle? Por varias razones:

- Evitar respuestas excesivamente largas
- Prevenir fugas accidentales de información sensible que el modelo pueda haber procesado
- Mantener la experiencia de usuario fluida
- Evitar explicaciones post-hoc que distraen del objetivo principal

Diferencias clave con los LLM tradicionales

Aunque comparten la misma base arquitectónica, los modelos de razonamiento se distinguen en cinco aspectos técnicos fundamentales:

Aspecto	LLM Tradicional	Modelo de Razonamiento
Entrenamiento	Premia la respuesta final correcta	Premia el proceso: pasos válidos, verificaciones, coherencia lógica
Cómputo en inferencia	Una sola pasada de generación, mismo costo para todas las consultas	Cómputo variable según dificultad: más pasos para problemas complejos
Uso de herramientas	Externo o poco integrado	Integrado en el ciclo de razonamiento
Explicaciones	Generadas después, pueden ser reconstrucciones	La traza de pensamiento se genera durante el proceso
Adaptación a dificultad	Mismo esfuerzo para todo	Sistema de 'gating': ajusta esfuerzo según necesidad

¿Cuándo usar cada tipo de modelo?

La elección entre un modelo tradicional y uno de razonamiento depende de la naturaleza de la tarea. No existe un modelo "mejor" en términos absolutos; cada uno tiene su espacio óptimo de aplicación.

Situaciones ideales para modelos de razonamiento

- Matemáticas y cálculos complejos: Problemas con múltiples pasos, conversiones de unidades, verificación de resultados
- Programación y debugging: Escribir código complejo, encontrar errores lógicos, proponer refactorizaciones
- Análisis de documentos: Detectar contradicciones, extraer información con validación cruzada, análisis legal o contractual
- Planificación: Crear planes de proyecto, identificar dependencias, proponer estrategias con evaluación de alternativas
- Razonamiento lógico: Puzzles, problemas de lógica formal, inferencias multi-paso

Situaciones ideales para LLM tradicionales

- Redacción creativa: Escribir correos, posts, contenido de marketing
- Conversación casual: Chat general, lluvia de ideas inicial, preguntas simples
- Resúmenes directos: Condensar textos sin análisis profundo
- Generación rápida: Cuando la velocidad es prioritaria y la precisión absoluta no es crítica
- Tareas rutinarias: Clasificación simple, extracción básica de información

Los trade-offs a considerar

Los modelos de razonamiento ofrecen mayor precisión, pero a un costo:

- **Latencia:** El tiempo de respuesta es significativamente mayor. Una consulta simple puede responderse en 1-2 segundos con un LLM tradicional, pero toma 10-30 segundos con un razonador.
- **Costo computacional:** Cada paso de razonamiento consume tokens y recursos. En entornos de producción, esto se traduce en costos operativos más altos.
- **Complejidad de uso:** Requiere entender cuándo activar el modo de razonamiento y con qué intensidad.

Agentes de Inteligencia Artificial:

Definición. Memoria de corto y largo plazo. Herramientas y funciones. MCP (Model Context Protocol)

1. Definición de Agente de Inteligencia Artificial

Un agente de inteligencia artificial representa un salto cualitativo respecto a los sistemas conversacionales tradicionales o chatbots simples. Mientras que estos últimos se limitan a responder preguntas basándose en patrones predefinidos o información contextual inmediata, un agente de IA constituye un sistema complejo que integra capacidades de razonamiento, planificación, memoria y acción para alcanzar objetivos de manera semi-autónoma. En su esencia, un agente utiliza un modelo de lenguaje grande (LLM) como motor de razonamiento, pero lo complementa con componentes adicionales que le permiten interactuar con el mundo exterior y mantener continuidad en sus operaciones.

La arquitectura fundamental de un agente puede conceptualizarse como la combinación sinérgica de cuatro elementos clave. El primero es el modelo de lenguaje, que actúa como el cerebro del sistema, proporcionando la capacidad de razonamiento y comprensión. El segundo componente es la memoria, que permite al agente mantener contexto tanto en el corto como en el largo plazo, recordando interacciones previas, preferencias del usuario y conocimientos adquiridos. El tercer elemento son las herramientas, que representan las extremidades del agente, permitiéndole interactuar con APIs externas, bases de datos, navegadores web y otros sistemas. Finalmente, el bucle de control coordina estos elementos, planificando acciones, observando resultados y ajustando el curso de acción según sea necesario.

Esta arquitectura se popularizó significativamente con la introducción del patrón ReAct (Reasoning and Acting), que propone un ciclo iterativo donde el agente

alterna entre fases de razonamiento y acción. En la fase de razonamiento, el agente analiza la situación actual, considera las opciones disponibles y decide el próximo paso. Luego, en la fase de acción, ejecuta una herramienta o función específica. Finalmente, observa el resultado de su acción y utiliza esta nueva información para continuar el razonamiento. Este patrón de "pensar, actuar y observar" permite que los agentes descompongan problemas complejos en pasos manejables y se adapten dinámicamente ante resultados inesperados.

Los casos de uso de agentes de IA son diversos y continúan expandiéndose. En el ámbito empresarial, encontramos asistentes de soporte técnico que pueden consultar múltiples sistemas corporativos simultáneamente para resolver tickets complejos, navegando entre documentación, bases de conocimiento y registros de clientes. En el desarrollo de software, los co-pilotos de programación funcionan como agentes que interactúan con repositorios de código, sistemas de integración continua y documentación técnica. En el análisis de datos, los agentes pueden consultar data warehouses, ejecutar consultas complejas y generar visualizaciones. Además, en automatización empresarial, estos sistemas integran calendarios, sistemas CRM, herramientas de comunicación y repositorios documentales para orquestar flujos de trabajo sofisticados.

2. Memoria en Agentes: Corto y Largo Plazo

2.1 Memoria de Corto Plazo

La memoria de corto plazo, también conocida como memoria de trabajo o working memory, constituye el buffer inmediato que el agente mantiene activo durante una sesión de trabajo. Esta memoria corresponde esencialmente al

context window del modelo de lenguaje, es decir, la ventana de tokens que el modelo puede procesar simultáneamente para generar respuestas coherentes. En términos prácticos, la memoria de corto plazo contiene las últimas N interacciones relevantes del usuario con el agente, las instrucciones vigentes, el estado actual de las sub-tareas en curso y cualquier información temporal necesaria para completar la tarea inmediata.

El propósito fundamental de la memoria de corto plazo es mantener coherencia local en la conversación. Permite que el agente comprenda referencias anafóricas ("eso que mencionaste antes"), mantenga el hilo de razonamientos multi-paso y recuerde decisiones tomadas momentos atrás. Sin embargo, esta memoria enfrenta limitaciones inherentes: los modelos de lenguaje tienen un límite finito en el tamaño de su context window, medido típicamente en miles de tokens. Cuando una conversación se extiende más allá de este límite, es necesario implementar estrategias de gestión.

La implementación más común de memoria de corto plazo utiliza buffers FIFO (First In, First Out), donde los mensajes más antiguos se descartan gradualmente a medida que se incorporan nuevos mensajes. Alternativamente, algunos sistemas implementan estrategias de archivado donde la información descartada se sintetiza y se transfiere a sistemas de memoria de largo plazo. Otras aproximaciones más sofisticadas emplean técnicas de compresión semántica, donde múltiples mensajes se resumen en representaciones más compactas que preservan la información esencial.

2.2 Memoria de Largo Plazo

La memoria de largo plazo representa el sistema de almacenamiento persistente que permite al agente recordar información más allá de sesiones individuales. Este tipo de memoria es fundamental para construir agentes verdaderamente

personalizados y eficientes, ya que les permite acumular conocimiento sobre las preferencias del usuario, recordar decisiones importantes tomadas en el pasado y aplicar aprendizajes de interacciones anteriores a situaciones nuevas. A diferencia de la memoria de corto plazo, que es volátil y limitada, la memoria de largo plazo está diseñada para ser duradera y escalable.

La implementación de memoria de largo plazo típicamente combina diferentes tecnologías de almacenamiento. Las bases de datos relacionales tradicionales pueden utilizarse para almacenar información estructurada como preferencias explícitas del usuario, configuraciones del agente o registros de transacciones. Sin embargo, una de las innovaciones más significativas en este campo es el uso de bases de datos vectoriales. Estas almacenan embeddings (representaciones vectoriales) de textos, permitiendo recuperación por similitud semántica. Cuando el agente necesita recordar información relevante, puede buscar en su memoria de largo plazo utilizando la similitud entre embeddings, recuperando no sólo coincidencias exactas sino también contenido conceptualmente relacionado.

3. Herramientas y Funciones: La Capacidad de Acción

Las herramientas y funciones representan el mecanismo que transforma a un modelo de lenguaje de un sistema puramente conversacional en un agente capaz de actuar en el mundo real. El concepto de tool calling o function calling se refiere a la capacidad del modelo de lenguaje de no solo generar texto como respuesta, sino de identificar cuándo necesita invocar una función externa, determinar qué función específica debe llamar, completar los parámetros necesarios en un formato estructurado (típicamente JSON) y devolver esta

invocación al runtime para su ejecución. Una vez ejecutada la función real, el resultado se retorna al modelo para que continúe su razonamiento y eventualmente formule una respuesta al usuario.

Este patrón ha sido adoptado ampliamente por los principales proveedores de modelos de lenguaje. OpenAI lo introdujo con su API de function calling, Google lo implementó en Gemini con su sistema de tool use, Anthropic lo incorporó en Claude, y Microsoft lo ofrece a través de Azure OpenAI. Aunque los detalles de implementación varían ligeramente entre proveedores, el concepto fundamental permanece constante: el modelo actúa como un orquestador inteligente que decide cuándo delegar tareas a sistemas especializados.

3.1 Casos de Uso de Herramientas

Los casos de uso de function calling abarcan un espectro amplio de aplicaciones. En el dominio de la augmentación de conocimiento, las herramientas permiten al agente consultar información actualizada o específica del dominio que no está presente en los datos de entrenamiento del modelo. Esto incluye consultas a APIs corporativas para obtener información de inventario, datos meteorológicos en tiempo real, cotizaciones de bolsa, o información de clientes desde sistemas CRM. Esta capacidad es fundamental para construir agentes que trabajen con información dinámica y contextual.

Más allá de consultar información, las herramientas permiten a los agentes realizar acciones concretas en sistemas externos. Un agente puede crear tickets en sistemas de gestión de proyectos, enviar correos electrónicos, programar reuniones en calendarios, actualizar registros en bases de datos, o incluso operar un navegador web para interactuar con páginas como lo haría un usuario humano. En algunos casos avanzados, los agentes pueden ejecutar código en entornos controlados, permitiéndoles realizar cálculos complejos, procesar

datos o generar visualizaciones. La capacidad de composición es particularmente poderosa: un agente puede orquestar múltiples herramientas en secuencia, donde el resultado de una llamada informa la siguiente, siguiendo el patrón de plan, ejecución de tool, observación del resultado, y replantación.

4. Model Context Protocol (MCP)

El Model Context Protocol (MCP) representa un desarrollo significativo en la estandarización de cómo los agentes de IA se conectan con fuentes de datos y herramientas externas. MCP es un protocolo abierto y de código abierto que propone una arquitectura unificada para estas integraciones, abordando un problema fundamental en el ecosistema actual de IA: la proliferación de integraciones específicas entre cada proveedor de modelo y cada fuente de datos o herramienta. Sin un estándar común, cada nueva integración requiere desarrollo custom para cada combinación de modelo y sistema externo, resultando en una complejidad $N \times M$ que no escala eficientemente.

4.1 Arquitectura y Funcionamiento de MCP

MCP propone una arquitectura cliente-servidor que desacopla los agentes de IA de las fuentes de datos específicas. En esta arquitectura, el host o aplicación del agente incorpora un cliente MCP que sabe comunicarse usando el protocolo MCP. Por otro lado, diferentes sistemas (GitHub, Google Drive, Slack, bases de datos corporativas, etc.) pueden implementar servidores MCP que exponen sus capacidades a través del protocolo estándar. Cada servidor MCP puede ofrecer tres tipos de primitivas: recursos (datos o contenido consultable), prompts (plantillas o flujos de interacción predefinidos) y herramientas (funciones invocables que realizan acciones).

La analogía que mejor ilustra MCP es la del USB-C en el mundo del hardware. Así como USB-C proporciona un conector universal que funciona con múltiples dispositivos y eliminó la necesidad de cables específicos para cada dispositivo, MCP pretende ser el "conector universal" para herramientas de IA. Una vez que un sistema implementa un servidor MCP, ese sistema se vuelve accesible para cualquier agente que soporte MCP, sin necesidad de escribir adaptadores específicos para cada modelo de lenguaje o entorno de agente. Esta interoperabilidad reduce dramáticamente el esfuerzo de desarrollo y mantenimiento.

4.2 Beneficios y Valor Estratégico

Los beneficios de MCP operan en múltiples dimensiones. La interoperabilidad es el beneficio más inmediato: un desarrollador puede construir una integración una sola vez y utilizarla con diferentes modelos de lenguaje y entornos, ya sea Claude, otros LLMs, entornos de desarrollo integrado (IDEs) o aplicaciones de chat. Esta portabilidad no solo ahorra tiempo de desarrollo inicial, sino que también simplifica el mantenimiento a largo plazo, ya que actualizaciones y mejoras se realizan en un solo lugar y benefician a todos los consumidores del servidor MCP.

La escalabilidad es otro beneficio crucial. A medida que las organizaciones adoptan agentes de IA más ampliamente y necesitan integrar más sistemas, la complejidad de integraciones punto-a-punto se vuelve insostenible. MCP reduce esta complejidad de $O(N \times M)$ a $O(N + M)$, donde cada sistema necesita implementar el protocolo solo una vez. Esto no solo reduce costos de desarrollo, sino que también acelera el time-to-market para nuevas integraciones. Adicionalmente, el ecosistema alrededor de MCP está creciendo: ya existen múltiples servidores MCP preconstruidos para sistemas populares, y

SDKs oficiales en varios lenguajes de programación facilitan la implementación de nuevos servidores.

4.3 Visión a Futuro: La Web Agéntica

Más allá de los beneficios técnicos inmediatos, MCP representa una apuesta estratégica hacia un futuro de "web agéntico". Esta visión imagina un ecosistema donde agentes de diferentes organizaciones puedan interoperar y cooperar, similar a cómo los servicios web actuales se comunican a través de APIs HTTP estándar. En este futuro, un agente podría descubrir dinámicamente las capacidades de otros agentes o servicios, componerlas de manera inteligente y orquestar flujos de trabajo complejos que crucen fronteras organizacionales. MCP proporciona los cimientos técnicos para este tipo de interoperabilidad.

Grandes actores de la industria han expresado interés en MCP como estándar fundamental para este ecosistema emergente. La adopción de MCP por parte de múltiples proveedores y plataformas sugiere que estamos presenciando el inicio de una convergencia hacia estándares comunes en la infraestructura de agentes de IA. Similar a cómo HTTP y HTML formaron la base de la web moderna, protocolos como MCP podrían formar la base de una nueva capa de servicios agénticos que transforme fundamentalmente cómo construimos y desplegamos aplicaciones de inteligencia artificial.

Workflows Agenticos. Niveles de Autonomía. Casos de Aplicación

¿Qué son los Workflows Agénticos?

Un workflow agéntico es la serie estructurada de pasos que un agente ejecuta para completar tareas complejas, combinando razonamiento, planificación, ejecución de herramientas y reflexión. A diferencia de workflows tradicionales que siguen secuencias rígidas predefinidas, los workflows agénticos son dinámicos y adaptativos, permitiendo que el agente ajuste su curso de acción basándose en observaciones y resultados intermedios. Estos workflows han emergido como patrones probados que representan soluciones reutilizables a desafíos recurrentes en el diseño de sistemas agénticos, permitiendo a los desarrolladores seleccionar la arquitectura apropiada para cada tipo de problema balanceando complejidad, confiabilidad, latencia y autonomía.

Patrones Fundamentales de Workflows

Los patrones fundamentales de workflows agénticos representan arquitecturas comprobadas para organizar cómo los agentes abordan problemas. Los principales patrones son:

- Sequential (Encadenamiento): El output de una llamada al LLM alimenta directamente el input de la siguiente en una cadena lineal predefinida, apropiado para tareas descomponibles en pasos fijos sin necesidad de adaptación.
- ReAct (Reasoning and Acting): Ciclo iterativo que alterna entre razonamiento explícito sobre qué hacer, ejecución de una acción

mediante herramientas, y observación de resultados para ajustar el plan, ideal para problemas que requieren exploración adaptativa.

- **Reflection (Reflexión):** El agente evalúa críticamente sus propios outputs, genera retroalimentación y refina iterativamente su trabajo hasta alcanzar calidad satisfactoria, valioso para tareas creativas donde la calidad puede mejorarse mediante auto-crítica.
- **Planning (Planificación):** Un agente planificador descompone problemas complejos en sub-tareas que delega a agentes especializados, luego sintetiza resultados, reduciendo carga cognitiva y permitiendo la paralelización y especialización.
- **Router (Enrutamiento):** Un agente router clasifica inputs y los dirige a agentes especializados según el dominio, logrando separación de preocupaciones y permitiendo que cada especialista sea optimizado independientemente.
- **Parallelization (Ejecución Paralela):** Sub-tareas independientes se ejecutan simultáneamente por múltiples agentes, reduciendo tiempo total drásticamente cuando hay alta latencia por llamada individual.

Niveles de Autonomía

Los agentes exhiben diferentes niveles de autonomía según cuánta supervisión requieren:

- **Nivel 0 - Asistentes sin herramientas:** Chatbot avanzado sin acceso a herramientas externas, limitado a su conocimiento entrenado y contexto inmediato.
- **Nivel 1 - Herramientas de solo lectura:** Acceso a consultas y búsquedas pero sin capacidad de modificar estado, cada acción requiere confirmación humana explícita.
- **Nivel 2 - Semi-autónomos con acciones limitadas:** Ejecuta secuencias de acciones incluyendo operaciones de escritura dentro de dominios

específicos, acciones de bajo riesgo automáticas pero alto impacto requieren aprobación.

- Nivel 3 - Parcialmente autónomos multi-dominio: Opera con autonomía significativa a través de múltiples dominios, ejecuta planes multi-paso complejos con supervisión asíncrona mediante monitoreo y puntos de control estratégicos.
- Nivel 4 - Completamente autónomos: Opera con supervisión mínima, establece sub-objetivos propios, adapta estrategias dinámicamente y puede crear nuevas herramientas según necesidad (actualmente experimental, mayoría de sistemas en producción operan en niveles 1-3).

Casos de Aplicación de Workflows Agénticos

Los workflows agénticos están transformando múltiples industrias al automatizar procesos complejos que anteriormente requerían inteligencia y juicio humano. Los siguientes casos ilustran cómo agentes operando bajo diferentes patrones y niveles de autonomía generan valor real en entornos de producción.

Soporte Técnico y Atención al Cliente

Agentes inteligentes manejan consultas de clientes clasificando el tipo de issue, accediendo a bases de conocimiento e historial del cliente, ejecutando diagnósticos automatizados, y resolviendo casos comunes sin intervención humana mientras escalan casos complejos a humanos con contexto completo ya recopilado. El valor generado incluye reducción del 60-80% en tiempo de primera respuesta, manejo automatizado del 40-60% de tickets, disponibilidad

24/7, y mayor satisfacción de agentes humanos al enfocarse en casos que requieren empatía y juicio complejo.

Análisis de Datos y Business Intelligence

Agentes analíticos permiten a stakeholders no técnicos formular preguntas en lenguaje natural y recibir análisis sofisticados. El agente descompone la pregunta, ejecuta queries SQL, genera visualizaciones apropiadas, analiza tendencias y segmentaciones, consulta datos externos para contexto, y sintetiza findings en narrativa coherente con recomendaciones. Esto democratiza el análisis: insights que requerían semanas de científicos de datos ahora se obtienen en minutos, acelerando la toma de decisiones y permitiendo que analistas de datos se enfoquen en problemas estratégicos complejos en lugar de consultas rutinarias.

Automatización de Procesos Empresariales

Agentes orquestan procesos end-to-end que abarcan múltiples sistemas corporativos. Por ejemplo, en onboarding de empleados, el agente crea cuentas en sistemas HR, aprovisiona accesos según rol, programa orientaciones, ordena equipo IT, asigna mentores, y envía comunicaciones personalizadas automáticamente. En procurement, valida políticas, verifica presupuestos, busca mejores precios, enruta aprobaciones apropiadamente, y coordina con vendedores una vez aprobado. El valor generado es reducción de procesos de días/semanas a horas/minutos, eliminación de errores humanos en tareas repetitivas, y liberación de empleados para trabajo estratégico de mayor valor.

Conclusión

Los agentes de IA, equipados con memoria, herramientas y protocolos estandarizados como MCP, alcanzan su máximo potencial cuando operan dentro de workflows bien diseñados. Los patrones fundamentales proporcionan arquitecturas probadas para diferentes tipos de problemas, mientras que los niveles de autonomía permiten balancear eficiencia con supervisión apropiada al contexto. Los casos de aplicación demuestran que workflows agénticos ya generan valor tangible en soporte al cliente, análisis de datos y automatización empresarial, con potencial creciente a medida que la tecnología madura.



Bibliografía utilizada y sugerida

- OpenAI. (2023). *Introducing ChatGPT and GPT models*. Disponible en: <https://openai.com>
- Devlin, J. et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. ACL.
- Vaswani, A. et al. (2017). *Attention Is All You Need*. NeurIPS.
- IBM. (2022). *Foundation Models in AI: A New Paradigm*. Disponible en: <https://www.ibm.com>
- Datacamp. (2023). *Interpretability and Bias in Large Language Models*. Disponible en: <https://www.datacamp.com>
- Kanaries. (2023). *Using AI for Creative Multimedia Production*. Disponible en: <https://docs.kanaries.net>